

First Writers-Readers Problem in a Microcontroller Report

German Becerra, Diya Jain, Dat Nguyen

Abstract:

The project aims to solve the writer-reader problem within the constraints of an ESP32 microcontroller to optimize sensor data acquisition and SD card logging for a rocket payload application. This report outlines the implementation strategy utilizing thread libraries (such as POSIX pthreads or esp_pthread.h), sensor modules (specifically BNO055), I2C communication for sensor data retrieval, and SDMMC/SPI Host APIs for efficient SD card data storage. The evaluation methodology involves statistical tests to assess system performance and safety for prolonged operation in the demanding environment of a rocket payload.

Intro:

Writer-reader problem (Version 1). Defined by Silberschatz, et.al as a variation of the readers-writers problem that requires that no reader be kept waiting unless a writer has already obtained permission to use the shared object. Therefore, it is a version that favors the reader. This will be implemented in a ESP32-WROOM-32D microcontroller, The main goal is to isolate the reading of different sensors along with the handling of their values and the writing of these values to an SD or MicroSD card. The goal of this program is to read from the sensors as fast as possible and log these values on the SD card to be handled later. The complete build will be part of a rocket payload and it will help to determine possible survivability metrics and the position of the rocket.

Methodology:

To tackle the writer-reader problem, the project relies on leveraging thread libraries suitable for the ESP32 environment. Extensive research informed the selection of POSIX pthreads or the esp_pthread.h library, chosen for their compatibility with the Espressif-IDF platform and their support for multi-threaded operations. These libraries facilitate concurrent execution of tasks, ensuring that sensor data reading and SD card writing occur efficiently without causing unnecessary delays.

Sensor modules play a pivotal role, with the BNO055 accelerometer chosen as the primary sensor due to its reliability and accuracy in capturing rocket dynamics. The project employs I2C communication to interface with the BNO055 and potentially other sensors, ensuring rapid data acquisition.

For persistent storage of sensor data, the project integrates the SPI Host APIs, accompanied by relevant SD association-provided freeware. The system optimizes data handling for the SD card, prioritizing speed and reliability to log sensor values promptly and accurately. Moreover, the project undertakes a comprehensive analysis of task turnaround times to fine-tune system performance and optimize resource allocation on the microcontroller.

Result:

The system was soldered and assembled with the said parts successfully, plus an additional LM2596 module for battery voltage regulation.

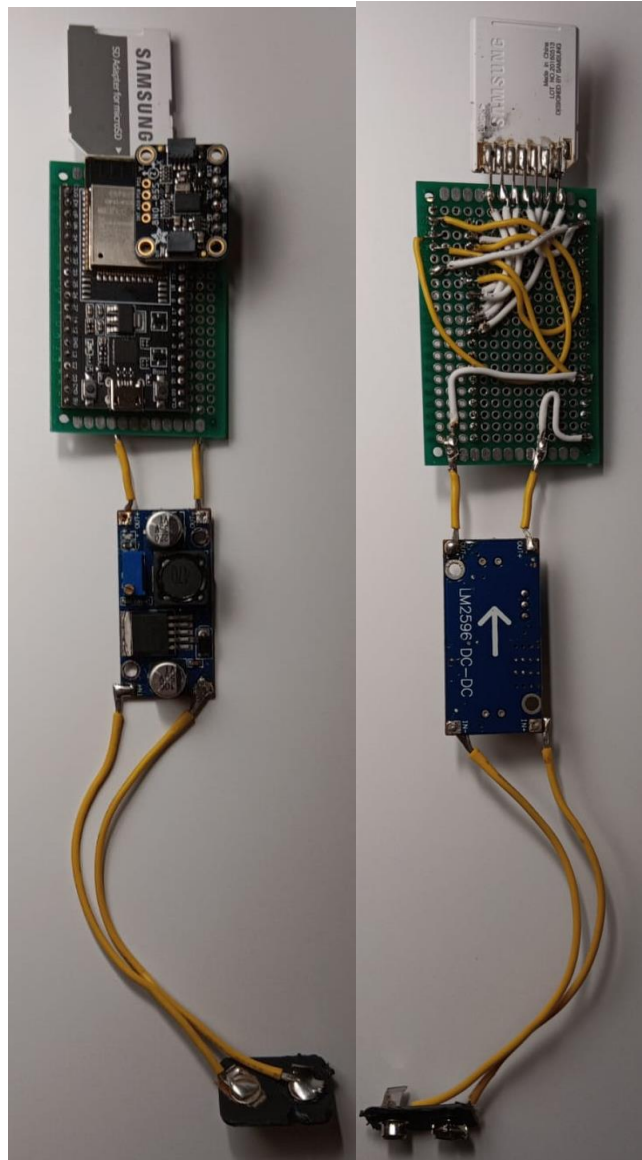


Figure 1: Front and back of physical system.

The complete code is attached as definitive.cpp, the main bulk of it contains code dealing with sensor fusion, gyroscopes, and linear algebra to obtain an input that is relevant from a rocket dynamics perspective, which is not relevant to this context. The following code box explains the main concept of the solution to this problem:

```

#define BUFFER_SIZE 100

SemaphoreHandle_t emptysema;
SemaphoreHandle_t full;
SemaphoreHandle_t mutex;

(Initialize semaphores and thread structures)

(DECLARE A LINKED LIST STRUCTURE, WITH THE METHOD clean() ACTING AS WRITER:)
void clean(){
    if(size < 10){ return; }
    NodeFuux<Xena> *arnolder = head;
    NodeFuux<Xena> *sandpaper;
    for(int n = 0; n < 5 && arnolder != NULL; n++){
        arnolder = arnolder->next;
    }

    if (arnolder == NULL || arnolder->next == NULL) {
        return;
    }

    NodeFuux<Xena> *Behelit = arnolder;
    while (Behelit->next != NULL) {
        sandpaper = Behelit->next;
        Printer Shalltear(sandpaper->data, type);

        xSemaphoreTake(emptysema, portMAX_DELAY);
        xSemaphoreTake(mutex, portMAX_DELAY);

        buffer[in] = Shalltear;
        in = (in+1) % BUFFER_SIZE;
        count++;

        xSemaphoreGive(mutex);
        xSemaphoreGive(full);

        Behelit->next = sandpaper->next;
        delete sandpaper;
        size--;
    }
}

(DECLARE THE FOLLOWING READER FUNCTION:)
void* consumer(void* arg){
    fs::FS fs = SD;

    File file = fs.open(LoggerFilename, FILE_APPEND);
    if (!file){while(1){}}

    xSemaphoreTake(full, portMAX_DELAY);
    xSemaphoreTake(mutex, portMAX_DELAY);

    Printer input = buffer[out];
    out = (out + 1) % BUFFER_SIZE;
    count--;

    xSemaphoreGive(mutex);
    xSemaphoreGive(emptysema);

    input.println(file);
}

file.close();
pthread_exit(NULL);
}

(THREAD WRITER AND READER AND DELETE SEMAPHORES AFTER THEY RETURN)

```

The complete code in definitive.cpp has extra error prevention and handling measures to avoid memory corruption, but still holds the same concept. Testing on the threaded system showed an increase of efficiency of about 158.9 %, this is attributed mainly to time gained not needing to constantly close and open the file to append a new value as this complete process is handled by the writer and to time the sensors can use to keep reading values instead of stopping while waiting to write. This time gain is subject to change due to the type of microSD or memory used.

Conclusion:

The implementation of the writer-reader problem solution on the ESP32 microcontroller demonstrates the feasibility of concurrently handling sensor data acquisition and SD card logging within the constraints of a rocket payload. The utilization of thread libraries, sensor modules, and communication protocols showcases the potential for achieving real-time, efficient data processing. Throughout the project, challenges related to timing synchronization and resource allocation were encountered, underscoring the complexities of real-time applications in such demanding environments. This experimentation not only addresses the writer-reader problem but also provides valuable insights into the complexities of deploying such systems in safety-critical environments. The project lays a solid groundwork for continued refinement and enhancement, ensuring the reliability and robustness required for rocket payload applications.